ESCOLA TÈCNICA SUPERIOR D'ENGINYERIA
Universitat Rovira i Virgili

INFORME TASK1

Enginyeria informàtica
URV curs 2025/2026
Pau Planas Carrasco
Adnan Echeki Hadouch

ÍNDICE

1. Introducción y Objetivos **3**
2. Entorno de Despliegue (Infraestructura AWS) **3**
3. Arquitectura 1: Comunicación Directa (NGINX + FastAPI) **4**
Resultados del Benchmark (Arquitectura Directa) 4
Análisis de Resultados (Arquitectura Directa) 6
4. Arquitectura Indirecta (Colas de Mensajes con RabbitMQ) **6**
Resultados del Benchmark (Arquitectura Indirecta) 7
Análisis de Resultados (Arquitectura Indirecta con RabbitMQ) 10
5. Correctitud y Consistencia de Datos (Gestión de Concurrencia) **11**
5.1. El problema de la Condición de Carrera (Race Condition) 11
5.2. Redis y la ejecución Atómica (Single-Thread) 11
5.3. Estrategia para Entradas No Numeradas: Comando DECR 11
5.4. Estrategia para Entradas Numeradas (Hotspots): Comando SETNX 11
5.5. Validación Empírica de los Resultados 12
6. Tolerancia a Fallos y Resiliencia del Sistema **12**
6.1. La fragilidad de la Arquitectura Síncrona (HTTP) 12
6.2. Resiliencia mediante Colas de Mensajes (RabbitMQ) 13
6.3. El mecanismo de Acknowledge Explícito (basic_ack) 13
6.4. Recuperación Automática (Requeueing) y Semántica de Entrega 13
7. Proceso Detallado de Despliegue en AWS **14**
7.1. Configuración de Red y Reglas de Seguridad (Security Groups) 14
7.2. Conexión a las Instancias mediante SSH (Git Bash) 14
7.3. Instalación de Dependencias (Gestor DNF) 14
7.4. Aprovisionamiento de Servicios Base 14
7.5. Puesta en Marcha y Ejecución de Pruebas 15
8. Conclusiones Finales **15**

1. Introducción y Objetivos

El objetivo principal de esta práctica ha sido diseñar, desplegar y analizar el rendimiento de dos arquitecturas de sistemas distribuidos preparadas para soportar una alta concurrencia de peticiones (simulando la venta masiva de entradas).

Se han comparado dos enfoques fundamentales:

1. **Arquitectura Directa (Síncrona):** Basada en peticiones HTTP REST utilizando NGINX como balanceador de carga y FastAPI.
2. **Arquitectura Indirecta (Asíncrona):** Basada en colas de mensajes utilizando RabbitMQ como *broker* intermedio.

El análisis se ha centrado en evaluar el rendimiento (peticiones por segundo), la correctitud de los datos (evitar vender más entradas de las disponibles) y la tolerancia a fallos.

2. Entorno de Despliegue (Infraestructura AWS)

Para realizar las pruebas en un entorno realista, se ha desplegado la infraestructura en la nube utilizando **Amazon Web Services (AWS)** a través de AWS Academy.

Se han utilizado dos instancias EC2 con el sistema operativo **Amazon Linux 2023**:

- **Máquina Servidor:** Encargada de alojar los servicios principales (Base de datos Redis, servidor web NGINX, *workers* de FastAPI y el *broker* RabbitMQ). Para garantizar la compatibilidad y el aislamiento de procesos, servicios como RabbitMQ y Redis se desplegaron mediante contenedores **Docker**.
- **Máquina Cliente:** Encargada de ejecutar los *scripts* de estrés (Productores/Clientes HTTP) para generar las avalanchas de peticiones.

Se configuraron los *Security Groups* (Firewall de AWS) habilitando los puertos necesarios para la comunicación entre las máquinas en la misma red virtual (VPC): puerto 80 (HTTP), 5672 (RabbitMQ) y 6379 (Redis).

3. Arquitectura 1: Comunicación Directa (NGINX + FastAPI)

En esta primera fase, se implementó una arquitectura cliente-servidor tradicional y síncrona.

Flujo de ejecución:

1. El Cliente lanza peticiones HTTP POST directamente al servidor.
2. Un servidor **NGINX** (escuchando en el puerto 80) actúa como balanceador de carga (*Load Balancer*) configurado con la directiva `upstream`.
3. NGINX reparte el tráfico mediante *Round-Robin* entre 3 *workers* de **FastAPI** (Uvicorn) ejecutándose en segundo plano en los puertos 8000, 8001 y 8002.

4. Los *workers* procesan la petición, se conectan a Redis, actualizan el inventario y devuelven una respuesta HTTP al cliente.

Resultados del Benchmark (Arquitectura Directa)

A continuación, se presentan los resultados obtenidos al lanzar las baterías de pruebas contra la API REST:

PRUEBAS EN LOCAL

Configuración	Test	Peticiones Enviadas	Éxitos	Rechazos	Tiempo Total	Rendimiento (pet/seg)
1 Worker (Sin NGINX)	No Numeradas	20.000	20.000	0	45.40 s	**440.51 pet/seg**
1 Worker (Sin NGINX)	Numeradas	25.997	20.000	5.997	57.31 s	**453.60 pet/seg**
3 Workers (Con NGINX)	No Numeradas	20.000	20.000	0	38.56 s	**518.61 pet/seg**
3 Workers (Con NGINX)	Numeradas	25.997	20.000	5.997	45.63 s	**569.69 pet/seg**
3 Workers (Con NGINX)	Hotspot	25.997	5.518	20.479	51.56 s	**504.23 pet/seg**

```
[ec2-user@ip-10-0-1-71 ~]$ python3 client.py
🧹 Base de datos limpiada.
=== TEST 1: ENTRADAS NO NUMERADAS ===
🚀 Iniciando envío de 20000 peticiones a http://10.0.1.185:80/buy/unnumbered...

--- 📊 RESULTADOS ---
✅ Compras exitosas: 20000
❌ Compras fallidas (rechazadas por BD): 0
⏱️ Tiempo total: 68.56 segundos
⚡ Rendimiento: 291.73 pet/seg

=== TEST 2: ENTRADAS NUMERADAS ===
🧹 Base de datos limpiada.
🚀 Iniciando envío de 25997 peticiones a http://10.0.1.185:80/buy/numbered...

--- 📊 RESULTADOS ---
✅ Compras exitosas: 20000
❌ Compras fallidas (rechazadas por BD): 5997
⏱️ Tiempo total: 92.04 segundos
⚡ Rendimiento: 282.46 pet/seg

=== TEST 3: ENTRADAS NUMERADAS (HOTSPOT) 🔥 ===
🧹 Base de datos limpiada.
🚀 Iniciando envío de 25997 peticiones a http://10.0.1.185:80/buy/numbered...

--- 📊 RESULTADOS ---
✅ Compras exitosas: 5518
❌ Compras fallidas (rechazadas por BD): 20479
⏱️ Tiempo total: 90.36 segundos
⚡ Rendimiento: 287.72 pet/seg

[ec2-user@ip-10-0-1-71 ~]$
```

PRUEBAS EN AWS

Test	Peticiones Enviadas	Éxitos	Rechazos	Tiempo Total	Rendimiento (pet/seg)
No Numeradas	20.000	20.000	0	68.56 s	**291.73 pet/seg**
Numeradas	25.997	20.000	5.997	92.04 s	**282.46 pet/seg**
Hotspot	25.997	5.518	20.479	90.36 s	**287.72 pet/seg**

Análisis de Resultados (Arquitectura Directa)

Tras analizar los datos obtenidos de la arquitectura síncrona (FastAPI), comparando su ejecución en local frente al despliegue en AWS con NGINX, destacamos las siguientes observaciones:

- **Escalabilidad Horizontal vs. Latencia de Red:** En el entorno local, añadir el balanceador NGINX y pasar de 1 a 3 *workers* demostró una mejora clara, subiendo el rendimiento de ~450 peticiones/segundo a picos de **~569 pet/seg**. Sin embargo, al desplegar esta misma configuración ganadora en AWS, el rendimiento se desplomó a la mitad (**~280 - 291 pet/seg**). Esto evidencia que, en entornos reales, el cuello de botella no es la capacidad de procesamiento del servidor, sino la **latencia de la red**.
- **El problema del modelo bloqueante (HTTP):** La caída de rendimiento en AWS se explica por la naturaleza del protocolo HTTP. El cliente debe abrir una conexión, enviar la petición, esperar a que viaje por internet, que el servidor la procese y que la respuesta vuelva. Multiplicar este bloqueo por 26.000 peticiones simultáneas provocó que el sistema tardara entre **68 y 92 segundos** en liberar completamente al cliente.
- **Correctitud garantizada (Test Hotspot):** A pesar de la lentitud introducida por la red, la arquitectura demostró ser completamente segura frente a condiciones de carrera. En el escenario más crítico (*Hotspot*), el sistema procesó 25.997 intentos de compra sobre asientos muy demandados. Gracias a la atomicidad de Redis, el sistema solo permitió **5.518 éxitos y rechazó correctamente 20.479 intentos** superpuestos, garantizando la integridad de los datos sin vender entradas de más, aunque penalizando el tiempo total (90.36 s en AWS frente a los 51.56 s en local).

Conclusión parcial: La arquitectura directa cumple su función de forma segura y consistente, beneficiándose del balanceo de carga con NGINX. Sin embargo, su sensibilidad a la latencia de red y su modelo de comunicación bloqueante la hacen ineficiente para soportar avalanchas de tráfico masivo (ventas de *ticketing*). Esta limitación hace indispensable la transición hacia un modelo asíncrono y desacoplado como RabbitMQ (Punto 4).

4. Arquitectura Indirecta (Colas de Mensajes con RabbitMQ)

Para la segunda fase, se modificó el sistema eliminando la comunicación directa entre el cliente y el servidor de procesamiento, introduciendo **RabbitMQ** como intermediario (*Message Broker*).

Flujo de ejecución:

1. El **Productor** (Cliente) genera las peticiones y las envía masivamente a través del protocolo AMQP a las colas de RabbitMQ (`cola_unnumbered`, `cola_numbered`, etc.).
2. RabbitMQ almacena los mensajes en memoria. El Productor no espera a que se procese la venta, solo a que RabbitMQ confirme la recepción.

3. El **Consumidor** (Worker automático), compuesto por un script en Python usando la librería `pika`, extrae los mensajes de la cola de forma asíncrona, se conecta a Redis, realiza la venta y confirma (*ack*) el mensaje.

Resultados del Benchmark (Arquitectura Indirecta)

A continuación, se muestran los resultados al inyectar los mensajes en RabbitMQ y procesarlos posteriormente:

```
[ec2-user@ip-10-0-1-71 ~]$ python3 producer.py
=== 1. ENTRADAS NO NUMERADAS ===
🚀 Metiendo 20000 mensajes en 'cola_unnumbered'...
✅ ¡Enviados en 1.59 segundos!

=== 2. ENTRADAS NUMERADAS (NORMAL) ===
🚀 Metiendo 25997 mensajes en 'cola_numbered'...
✅ ¡Enviados en 2.05 segundos!

=== 3. ENTRADAS NUMERADAS (HOTSPOT 🔥) ===
🚀 Metiendo 25997 mensajes en 'cola_hotspot'...
✅ ¡Enviados en 2.10 segundos!

[ec2-user@ip-10-0-1-71 ~]$ python3 producer.py
=== 1. ENTRADAS NO NUMERADAS ===
🚀 Metiendo 20000 mensajes en 'cola_unnumbered'...
✅ ¡Enviados en 1.64 segundos!

=== 2. ENTRADAS NUMERADAS (NORMAL) ===
🚀 Metiendo 25997 mensajes en 'cola_numbered'...
✅ ¡Enviados en 2.00 segundos!

=== 3. ENTRADAS NUMERADAS (HOTSPOT 🔥) ===
🚀 Metiendo 25997 mensajes en 'cola_hotspot'...
✅ ¡Enviados en 2.06 segundos!

[ec2-user@ip-10-0-1-71 ~]$
```

PRUEBAS EN AWS

Ejecución	Test	Mensajes Enviados	Tiempo Total	Rendimiento (msg/seg)
1	**No Numeradas**	20.000	1.59 s	~12.578 msg/seg
1	**Numeradas (Normal)**	25.997	2.05 s	~12.681 msg/seg

1	**Hotspot**	25.997	2.10 s	~12.379 msg/seg
2	**No Numeradas**	20.000	1.64 s	~12.195 msg/seg
2	**Numeradas (Normal)**	25.997	2.00 s	~12.998 msg/seg
2	**Hotspot**	25.997	2.06 s	~12.619 msg/seg

```
🚀 ¡Avalancha de 632 mensajes en 'cola_unnumbered'!
🧹 Base de datos reseteada automáticamente.
⚙️ Procesados: 5000...
⚙️ Procesados: 10000...
⚙️ Procesados: 15000...
⚙️ Procesados: 20000...
✅ TEST COMPLETADO: cola_unnumbered
📊 Exitos: 20000 | Fallos: 0
⏱️ Tiempo: 10.30 s
⚡ Rendimiento: 1942.00 pet/seg
----------------------------------------

🚀 ¡Avalancha de 25997 mensajes en 'cola_numbered'!
🧹 Base de datos reseteada automáticamente.
⚙️ Procesados: 5000...
⚙️ Procesados: 10000...
⚙️ Procesados: 15000...
⚙️ Procesados: 20000...
⚙️ Procesados: 25000...
✅ TEST COMPLETADO: cola_numbered
📊 Exitos: 20000 | Fallos: 5997
⏱️ Tiempo: 13.56 s
⚡ Rendimiento: 1916.81 pet/seg
----------------------------------------

🚀 ¡Avalancha de 25997 mensajes en 'cola_hotspot'!
🧹 Base de datos reseteada automáticamente.
⚙️ Procesados: 5000...
⚙️ Procesados: 10000...
⚙️ Procesados: 15000...
⚙️ Procesados: 20000...
⚙️ Procesados: 25000...
✅ TEST COMPLETADO: cola_hotspot
📊 Exitos: 5518 | Fallos: 20479
⏱️ Tiempo: 16.31 s
⚡ Rendimiento: 1594.39 pet/seg
----------------------------------------
```

PRUEBAS DE AWS

Test	Peticiones Procesadas	Éxitos	Fallos (Rechazadas)	Tiempo Total	Rendimiento (pet/seg)
No Numeradas	20.000	20.000	0	10.30 s	1942.00 pet/seg
Numeradas	25.997	20.000	5.997	13.56 s	1916.81 pet/seg
Hotspot	25.997	5.518	20.479	16.31 s	1594.39 pet/seg

PRUEBAS EN LOCAL

Test	Peticiones Procesadas	Éxitos	Fallos (Rechazadas)	Tiempo Total	Rendimiento (pet/seg)
No Numeradas	**20.000**	**20.000**	**0**	**12.13 s**	**1648.61 pet/seg**
Numeradas	**25.997**	**20.000**	**5.997**	**13.93 s**	**1865.94 pet/seg**

Hotspot	**25.997**	**5.518**	**20.479**	**13.59 s**	**1913.14 pet/seg**

Análisis de Resultados (Arquitectura Indirecta con RabbitMQ)

Tras analizar los datos de la arquitectura asíncrona (RabbitMQ + Consumidores), tanto en local como en AWS, destacamos las siguientes observaciones:

- **Salto masivo de rendimiento (Asincronía):** El sistema pasa de procesar ~500 peticiones/segundo (con NGINX) a unas espectaculares **~1.600 - 1.900 peticiones/segundo**. Al eliminar la espera de la respuesta HTTP, el Productor inyecta los mensajes instantáneamente en la cola y el Consumidor los procesa a la máxima velocidad que permite la CPU y la base de datos.
- **Absorción de picos de carga (Buffering):** Las colas actúan como un "amortiguador" perfecto. Ante la avalancha de casi 26.000 peticiones simultáneas, el sistema no sufre caídas ni bloqueos de red (*timeouts*). Los mensajes esperan pacientemente en memoria hasta que el *worker* está libre para procesarlos.
- **Gestión impecable de colisiones (Hotspot):** En el escenario más crítico (*Hotspot*), donde todos los usuarios compiten por los mismos asientos, la arquitectura mantiene el tipo procesando el lote completo en apenas **13 - 16 segundos**. Rechaza correctamente las más de 20.000 sobreventas sin que los bloqueos atómicos de Redis penalicen drásticamente el rendimiento global.

Conclusión parcial: La arquitectura indirecta basada en colas de mensajes demuestra ser inmensamente superior para este caso de uso. Resuelve el problema del cuello de botella de la red, garantiza la integridad de los datos (ACID) y ofrece una escalabilidad y tolerancia a fallos que la arquitectura síncrona simplemente no puede alcanzar.

5. Correctitud y Consistencia de Datos (Gestión de Concurrencia)

Uno de los requisitos más críticos y complejos en el diseño de este sistema distribuido era garantizar la **consistencia estricta de los datos**. En un escenario de alta demanda (*ticketing*), el sistema debe asegurar de forma absoluta que no se vendan más de **20.000 entradas** bajo ninguna circunstancia, evitando el temido problema de la sobreventa (*overselling*).

5.1. El problema de la Condición de Carrera (*Race Condition*)

Al contar con múltiples *workers* (ya sean procesos de FastAPI con NGINX o consumidores asíncronos de RabbitMQ) operando en paralelo, existe un alto riesgo de colisión. Si se utilizara un enfoque tradicional de lectura-escritura (leer el inventario -> restar 1 -> guardar

el inventario), dos *workers* podrían leer simultáneamente que queda "1 entrada disponible", procesar la venta y dejar el inventario en números negativos. Para evitar la necesidad de implementar costosos bloqueos de base de datos (*locks*) que hundirían el rendimiento del sistema, se optó por una solución basada en **operaciones atómicas**.

5.2. Redis y la ejecución Atómica (Single-Thread)

Para gestionar el estado global del inventario, se delegó la responsabilidad a **Redis**. La principal ventaja arquitectónica de Redis para este caso de uso es su modelo de procesamiento *Single-Threaded* (de un solo hilo) para la ejecución de comandos. Al encolar y ejecutar las peticiones secuencialmente a nivel interno, Redis garantiza que cualquier comando que modifique un dato sea **nativamente atómico**. Esto elimina de raíz la posibilidad de que dos *workers* alteren el mismo registro en el mismo milisegundo.

Para resolver los distintos escenarios de venta, se utilizaron dos estrategias algorítmicas distintas en la base de datos:

5.3. Estrategia para Entradas No Numeradas: Comando `DECR`

Para el control del volumen total de entradas de acceso general, el inventario se inicializa con el valor máximo (20.000). Los *workers* no leen el valor previo, sino que lanzan directamente el comando atómico `DECR` (decremento).

- La base de datos resta 1 al contador y devuelve el nuevo valor en una única operación indivisible.
- Si el valor devuelto por Redis es `>= 0`, el *worker* considera la venta como válida.
- Si el valor devuelto es `< 0`, el *worker* sabe que el inventario se ha agotado y rechaza la transacción, impidiendo la sobreventa.

5.4. Estrategia para Entradas Numeradas (Hotspots): Comando `SETNX`

El escenario más hostil en la venta de entradas ocurre cuando miles de usuarios intentan adquirir exactamente la misma butaca (*Hotspot*). Para evitar duplicados en los asientos numerados, se utilizó el comando `SETNX` (*Set if Not eXists*). En este caso, la clave en Redis representa el número de asiento (ej. `asiento:45`). Cuando varios *workers* intentan asignar ese asiento a distintos clientes concurrentemente, lanzan el comando `SETNX`.

- Redis garantiza que **solo el primer comando en llegar** insertará el valor y recibirá un `1` (éxito).
- Las fracciones de milisegundo posteriores, el resto de intentos recibirán un `0` (fallo), y el *worker* rechazará la petición indicando que el asiento ya está ocupado.

5.5. Validación Empírica de los Resultados

Los resultados empíricos expuestos en las tablas anteriores (Puntos 3 y 4) demuestran el éxito total de esta implementación. En las pruebas de entradas no numeradas, ante

avalanchas superiores a 25.000 peticiones, el sistema cortó de forma quirúrgica exactamente en **20.000 éxitos**, sin un solo falso positivo.

Aún más destacable es la prueba de **Hotspot**, donde se simularon colisiones masivas de usuarios intentando comprar los mismos asientos. El sistema gestionó las peticiones concurrentes a la perfección, logrando aislar y validar únicamente **5.518 compras únicas exitosas** y rechazando de forma segura y consistente las **20.479 peticiones duplicadas** que habrían provocado inconsistencia de datos en un sistema tradicional.

6. Tolerancia a Fallos y Resiliencia del Sistema

Un aspecto crítico evaluado durante esta práctica ha sido la capacidad de la infraestructura para recuperarse ante fallos inesperados en los nodos de procesamiento (*workers*), garantizando que no se pierdan las peticiones de los usuarios en caso de caída del servidor o cortes de red. En este aspecto, la diferencia entre ambos paradigmas arquitectónicos es radical.

6.1. La fragilidad de la Arquitectura Síncrona (HTTP)

En la Arquitectura Directa (NGINX + FastAPI), el ciclo de vida de una transacción está fuertemente acoplado a la conexión TCP viva entre el cliente y el servidor. Si un *worker* falla (por ejemplo, debido a un desbordamiento de memoria, un reinicio inesperado o un corte de red) justo en el momento en que está procesando la compra en Redis, la conexión HTTP se rompe abruptamente. Como consecuencia:

1. El cliente recibe un error genérico (ej. *502 Bad Gateway* o un *Timeout*).
2. El estado de la compra queda en el limbo (el cliente no sabe si se cobró o no).
3. **La petición se pierde irremediablemente**, obligando al usuario a reintentar el proceso desde cero (con el riesgo de que las entradas ya se hayan agotado).

6.2. Resiliencia mediante Colas de Mensajes (RabbitMQ)

Para solucionar esta debilidad, la Arquitectura Indirecta introdujo una capa vital de resiliencia al delegar la custodia de los mensajes a RabbitMQ, actuando como un *buffer* seguro. El Productor simplemente deposita el mensaje en el *broker* y se desentiende, sabiendo que RabbitMQ se hace responsable de él.

6.3. El mecanismo de *Acknowledge* Explícito (`basic_ack`)

Para garantizar que ningún mensaje se pierda si un Consumidor falla, se implementó el patrón de **Confirmación Explícita de Mensajes** mediante el protocolo AMQP. El flujo diseñado funciona de la siguiente manera:

1. Cuando RabbitMQ entrega un mensaje a un *worker*, **no lo elimina de la cola**. En su lugar, lo marca con un estado interno de retención temporal llamado *Unacknowledged* (No confirmado).

2. El *worker* extrae los datos, se conecta a Redis y ejecuta la lógica de la compra (usando `DECR` o `SETNX`).
3. **Solo si la operación en la base de datos finaliza con éxito**, el *worker* envía una señal de `basic_ack` a RabbitMQ. Es en este preciso momento cuando el *broker* elimina permanentemente el mensaje de la memoria.

6.4. Recuperación Automática (*Requeueing*) y Semántica de Entrega

El verdadero poder de este diseño se manifiesta ante un fallo crítico. Si un *worker* se desconecta, se bloquea o sufre un *crash* a mitad del procesamiento (antes de enviar el *Ack*), la conexión de red entre ese proceso y RabbitMQ se cierra.

RabbitMQ detecta inmediatamente la pérdida del canal de comunicación y asume que el mensaje no ha sido procesado correctamente. Automáticamente, revoca el estado *Unacknowledged* y **vuelve a encolar el mensaje** (lo devuelve al estado *Ready*). De este modo, la petición es reasignada de manera transparente e inmediata al siguiente *worker* disponible en el clúster.

Conclusión parcial: Gracias al uso de colas y al mecanismo de *acknowledgment*, la arquitectura indirecta logra una semántica de entrega de **"Al menos una vez" (*At-least-once delivery*)**. El sistema asegura matemáticamente que ninguna intención de compra se perderá por fallos de infraestructura, proporcionando un grado de tolerancia a fallos empresarial que la arquitectura síncrona es incapaz de ofrecer.

7. Proceso Detallado de Despliegue en AWS

Para materializar las arquitecturas descritas en este informe, se llevó a cabo un proceso de aprovisionamiento y configuración en la plataforma AWS Academy. A continuación, se detallan las fases técnicas del despliegue sobre instancias EC2 con el sistema operativo Amazon Linux 2023.

7.1. Configuración de Red y Reglas de Seguridad (Security Groups)

El primer paso consistió en configurar el Firewall perimetral de AWS (*Security Group*). Para permitir la comunicación, se habilitaron las siguientes reglas de entrada (*Inbound Rules*): el puerto 22 (SSH), el puerto 80 (NGINX), el puerto 6379 (Redis), y los puertos 5672 y 15672 (RabbitMQ).

7.2. Conexión a las Instancias mediante SSH (Git Bash)

Para interactuar con la infraestructura en la nube, se utilizaron terminales de comandos locales basadas en **Git Bash**. El acceso a cada instancia (Servidor y Cliente) se realizó de forma segura introduciendo las credenciales y el comando SSH exacto proporcionado por el

panel de detalles de **AWS Academy** (ej. `ssh -i "claves-sd.pem" ec2-user@<IP_PUBLICA>`). Se mantuvieron múltiples terminales abiertas simultáneamente para poder administrar los *workers* y monitorizar los recursos en tiempo real.

7.3. Instalación de Dependencias (Gestor DNF)

Una vez establecida la conexión SSH con las máquinas de Amazon Linux 2023, la preparación del entorno se realizó utilizando **`dnf`**, el gestor de paquetes oficial y nativo de esta distribución. Mediante comandos como `sudo dnf install -y docker python3 nginx`, se descargaron e instalaron los cimientos del sistema. Posteriormente, se utilizó el gestor de Python (`pip`) para instalar las librerías específicas requeridas por el código: `fastapi`, `uvicorn`, `redis`, `requests` y `pika`.

7.4. Aprovisionamiento de Servicios Base

Con las herramientas instaladas mediante `dnf`, se procedió a levantar los servicios críticos en la máquina Servidor:

- **Docker:** Se utilizaron comandos `docker run` para desplegar la base de datos Redis (en el puerto 6379) y el clúster de RabbitMQ (puertos 5672/15672), garantizando un entorno limpio y aislado.
- **NGINX:** Se configuró el archivo `/etc/nginx/nginx.conf` creando un bloque `upstream` que enlazaba el puerto 80 con los puertos locales de FastAPI (8000, 8001, 8002) mediante balanceo *Round-Robin*.

7.5. Puesta en Marcha y Ejecución de Pruebas

Tras completar la instalación de dependencias y servicios, se procedió finalmente a la ejecución de las baterías de pruebas para ambas arquitecturas:

- **Prueba de Arquitectura Directa:** En la máquina Servidor, se abrieron tres sesiones en Git Bash para ejecutar simultáneamente los tres *workers* (`uvicorn worker:app --port 8000 &`, `8001` y `8002`). Una vez el servidor estaba a la escucha, desde la máquina Cliente se ejecutó el *script* `client.py` para lanzar las avalanchas HTTP contra NGINX.
- **Prueba de Arquitectura Indirecta:** Se detuvieron los procesos anteriores. En el Servidor, se ejecutó el *script* `consumer.py` para que el *worker* comenzara a vigilar las colas de RabbitMQ. Inmediatamente después, en la máquina Cliente, se ejecutó `producer.py` para inyectar masivamente los mensajes en el *broker* AMQP y registrar los tiempos de procesamiento.

8. Conclusiones Finales

Tras el despliegue, ejecución y análisis exhaustivo de ambas arquitecturas en la infraestructura de AWS, se extraen las siguientes conclusiones fundamentales:

- **1. Rendimiento y el Límite de la Red (Latencia):** La Arquitectura Indirecta (RabbitMQ) ha demostrado una superioridad aplastante para gestionar avalanchas de tráfico. Al desacoplar la recepción del procesamiento, se elimina el tiempo de espera bloqueante del protocolo HTTP. Empíricamente, el sistema asíncrono logró procesar en torno a las ~1.900 peticiones/seg, multiplicando casi por 6 el rendimiento de la solución directa con NGINX (~290 pet/seg), la cual se vio severamente penalizada por la latencia de red.
- **2. Correctitud y Consistencia (ACID):** Ambas arquitecturas han demostrado ser fiables a la hora de evitar la sobreventa garantizando la integridad de los datos. Sin embargo, en escenarios de máxima contención (*Hotspots*), la cola de mensajes absorbe el impacto de las colisiones concurrentes de manera mucho más eficiente, evitando que las conexiones de los clientes colapsen por *timeouts* mientras el motor *Single-Threaded* de Redis procesa la saturación.
- **3. Resiliencia y Tolerancia a Fallos:** La arquitectura síncrona es frágil frente a caídas de red; si un *worker* falla, la petición HTTP se pierde irremediablemente. Por el contrario, la introducción de RabbitMQ dota al sistema de una semántica de entrega *"At-least-once"*. Gracias a los *acknowledgments* explícitos (`basic_ack`), se asegura matemáticamente que ninguna intención de compra se pierda por fallos técnicos.
- **4. Trade-off entre Complejidad y Escalabilidad:** La solución con NGINX + FastAPI es indudablemente más sencilla de desplegar y mantener. No obstante, el sistema con RabbitMQ escala de manera horizontal de forma mucho más natural; basta con levantar nuevos contenedores de *workers* que se conecten a la cola para aumentar la velocidad de vaciado, sin necesidad de reconfigurar manualmente las rutas de un balanceador de carga.

Veredicto Definitivo: Para un sistema crítico de alta demanda como es el *ticketing*, donde los picos masivos son la norma y la pérdida de datos es inaceptable, la Arquitectura basada en Colas de Mensajes (Event-Driven) resulta ser indiscutiblemente la opción más robusta y escalable, justificando sobradamente su mayor complejidad inicial de infraestructura.